

IT5005 Artificial Intelligence

Tutorial 6 Suggested Solutions

Question 1

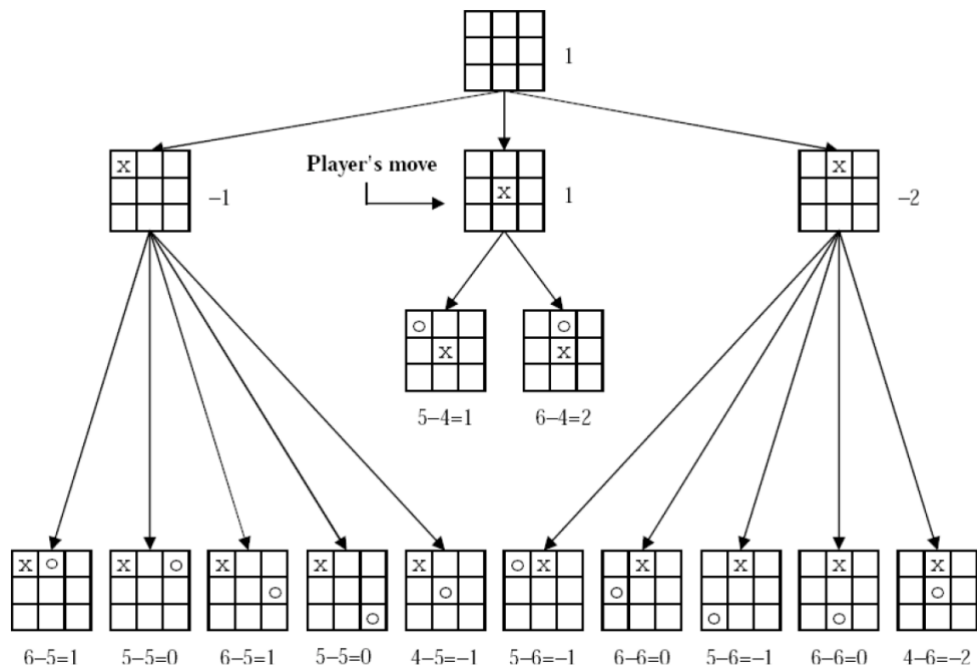
Consider the full Tic-Tac-Toe search space shown in Figure 1. The TicTac-Toe search space can actually be reduced by means of symmetry. This is done by eliminating those states which become identical with an earlier state after a symmetry operation (e.g. rotation). The following diagram shows a reduced state space for the first three levels with the player making the first move using "x" and the opponent making the next move with "o".

Assume that the following heuristic evaluation function is used at each leaf node n :

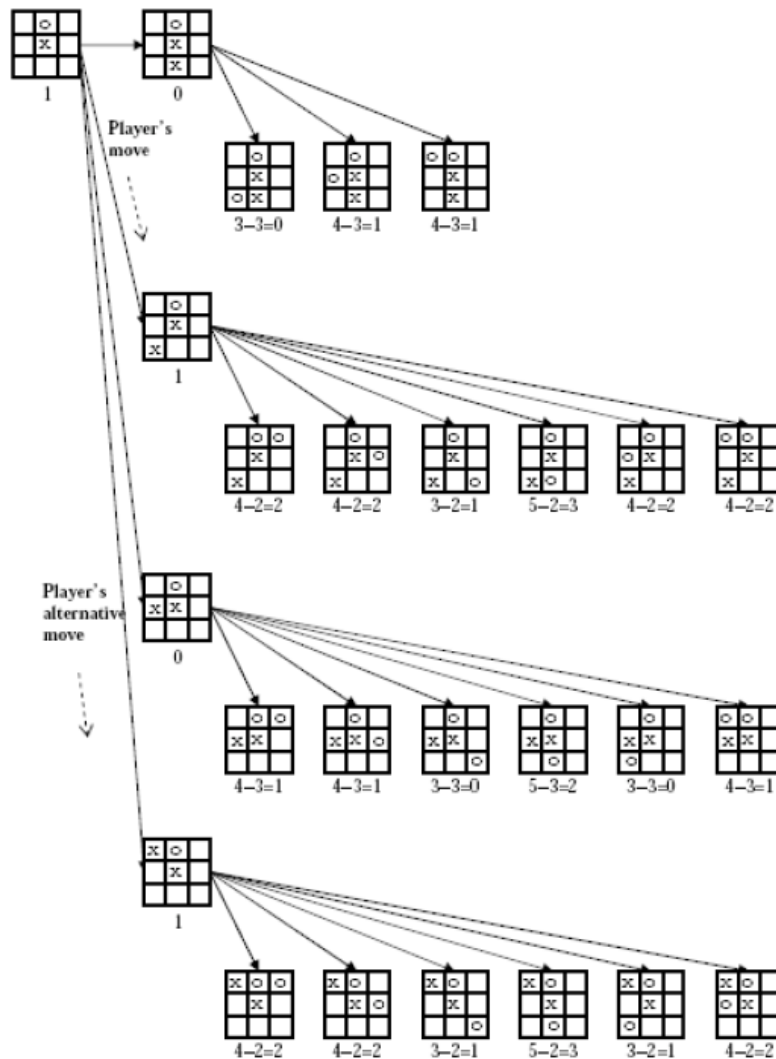
$$Eval(n) = P(n) - O(n)$$

where $P(n)$ is the number of winning lines for the player while $O(n)$ is the number of winning lines for the opponent. A winning line for the player is a line (horizontal, vertical or diagonal) that either contains nothing or "x". For the opponent, it is either nothing or "o" in the winning line. Thus, for the leftmost leaf node in Figure 2, $Eval(n) = 6 - 5 = 1$.

- (a) Use the minimax algorithm to determine the first move of the player, searching 2-ply deep search space shown in Figure 2.



1 Assume that the “x” player will now make his second move after his opponent has placed an “o”. Complete the following minimax tree in Figure 3 by filling the remaining blank boards at the leaf nodes. Compute the evaluation function for each of the filled leaf nodes and determine the second move of the “x” player (searching 2-ply deep).



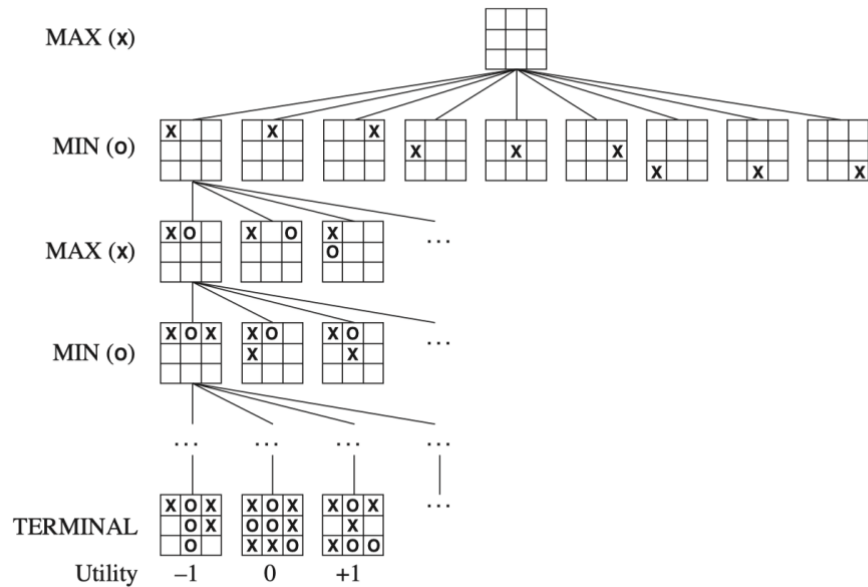
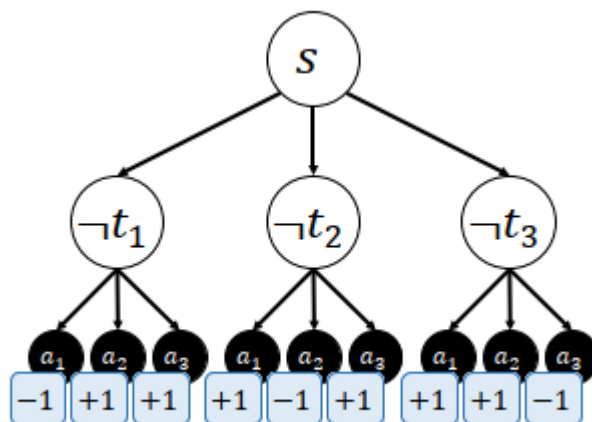


Figure 3: 3-ply deep search space

Question 2

Consider the following game: we have an attacker looking at three targets: t_1, t_2 and t_3 . A defender must choose which of the two targets it will guard; however, the attacker has an advantage: it can observe what the defender is doing before it chooses its move. If an attacker successfully attacks it receives a payoff of 1 and the defender gets a payoff of -1 .

- (a) Model this problem as a minimax search problem. Draw out the search tree. What is the defender's payoff in this game?



- (b) Can the defender do better by randomizing? What is the defender's optimal strategy? Prove your claim.

In the original game the attacker always has one path to attack to get a gain of 1, and the defender gets a gain of -1. However by randomizing protecting two of the three targets as the attacker launches the attack, the attacker will succeed 1/3 of the time and the defender will succeed 2/3 of the time. This gives an expected score of $1/3 \times -1 + 2/3 \times 1 = 1/3$

Question 3.

We have a 1-D Kohonen map with 5 units (neurons) is shown below.

0	1	2	3	4
(3, 9)	(17, 9)	(3, 23)	(42, 20)	(4, 21)

The format for each unit is:

<cell number> <2D vector containing current unit contents>
--

We are given the following input:

$X = (18, 7)$

- a. Using an L2 norm (a "norm" is just another name for distance) defined as:

$$l_2(X, Y) = \sum_{i=0}^{n-1} (x_i - y_i)^2$$

Compute the following table showing the distance between X and each of the units:

$X = (18, 7)$

Unit	Distance between X and unit
0 (3, 9)	229
1 (17, 9)	5
2 (3, 23)	481
3 (42, 20)	745
4 (4, 21)	392

Which is the best-matching unit? **Unit 1**

- b. Explain the purpose of the neighborhood function in Kohonen maps.

The neighborhood function changes units surrounding the BMU to look like the input vector, but to a lesser degree. This encourages vectors like are similar to the current input vector to cluster to nearby units, thus forming a map where similar vectors are clustered together.

- c. We have the following neighborhood function:

0.25	0.75	1	0.75	0.25
------	------	---	------	------

Using a training rate $\alpha = 0.7$, update the Kohonen map based on your results in part a. Write out the new map below:

Our update equation is:

$$n_i = n_i + \alpha \cdot h(i) \cdot (v_j - n_i)$$

The centre of the neighborhood function h (with the value of 1) corresponds to the mask multiplier for the BMU, while the “0.75” squares correspond to the immediate neighbours, and the “0.25” squares correspond to the neighbors of the neighbours of the BMU. Note that if the function exceeds the boundaries of the Kohonen map, it is simply ignored (i.e. we can imagine that it hangs over the edge of the map and does nothing).

The update equations for each unit (note: x_0 refers to the first coordinate of unit 0, y_0 refers to the second coordinate of unit 0, etc.)

Unit	Original Unit Values Input	Equation	New Value
1 (BMU)	(17, 9) (18, 7)	$x_1 = 17 + 0.7 * 1 * (18 - 17)$ $y_1 = 9 + 0.7 * 1 * (7 - 9)$	(17.7, 7.6)
0 (BMU)	(3, 9) (18, 7)	$x_0 = 3 + 0.7 * 0.75 * (18 - 3)$ $y_0 = 9 + 0.7 * 0.75 * (7 - 9)$	(10.9, 7.95)
2 (BMU)	(3, 23) (18, 7)	$x_2 = 3 + 0.7 * 0.75 * (18 - 3)$ $y_2 = 23 + 0.7 * 0.75 * (7 - 23)$	(10.4, 14.6)
3 (BMU)	(42, 20) (18, 7)	$x_3 = 42 + 0.7 * 0.25 * (18 - 42)$ $y_3 = 20 + 0.7 * 0.25 * (7 - 20)$	(37.8, 17.7)

New map:

0 (10.9, 7.95)	1 (17.7, 7.6)	2 (10.4, 14.6)	3 (37.8, 17.7)	4 (4, 21)
-------------------	------------------	-------------------	-------------------	--------------

Question 4.

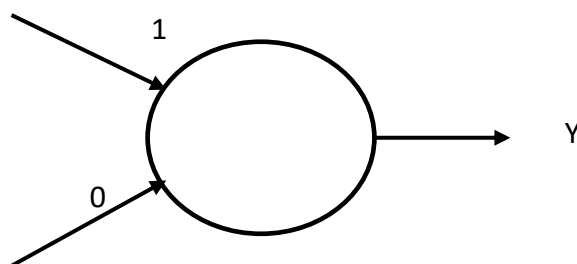
- a. What is the purpose of the bias in Perceptrons? Show, using a suitable example, the importance if this bias.

The Perceptron learns linear functions in the form of $y = a.x + b$, and the bias corresponds to the intercept b , allowing the function to move up and down the y axis. Without the bias the function can only rotate about the origin and can only learn linear functions of the form $y = ax$.

- b. Does the value of the bias matter? Why or why not?

Yes and no: No because the weight corresponding to the bias will just be adjusted accordingly to fit the underlying function that the Perceptron is trying to learn, yet at the same time if its absolute value is too large the network may not converge (or may converge to infinity).

- c. We have a 1-input Perceptron shown below, with a bias input of 1:



Given the following data to be learnt:

X	Y
2	6
6	14.5
4	10.75

Complete the following table:

The learning equation is $w_j = w_j + \alpha \cdot (y_i - y_{out}) \cdot x_{ij}$. Therefore we will calculate the error is $y_i - y_{out}$.

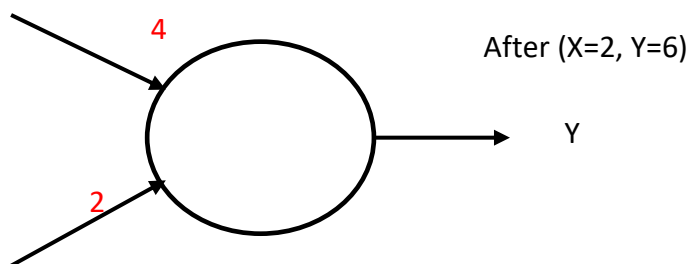
X	Y	Y_{out}	Error
2	6	3.3	2.7
6	14.5	8.3	6.2
4	10.75	5.8	4.95

- d. Using the results in part c., apply the Perceptron learning law to update the weights, using a learning rate of $\alpha = 0.6$. Write the new weights on the diagrams below:

The first row is the weight for X, the second row is the weight for the bias:

$$w_0 = 1.25 + 0.6 * 2.7 * 2$$

$$w_1 = 0.8 + 0.6 * 2.7 * 1$$



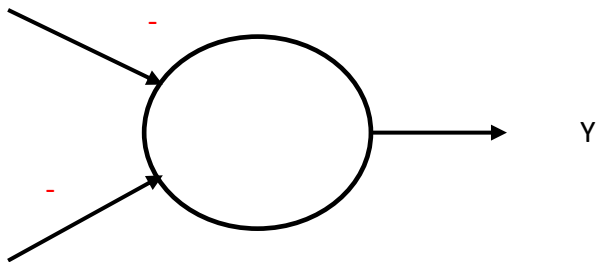
New errors after first sample:

X	Y	Y_{out}	Error
2	6	11.4	-5.4
6	14.5	29.36	-14.86
4	10.75	20.38	-9.63

After (X=6, Y=14.5)

$$w_0 = 4.49 + 0.6 * (-14.86) * 6$$

$$w_1 = 2.42 + 0.6 * (-14.86) * 1$$



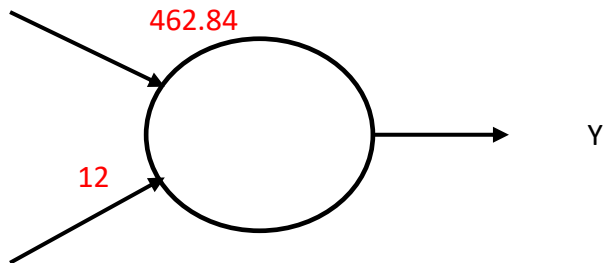
New errors after second sample:

X	Y	Y _{out}	Error
2	6	-104.508	110.508
6	14.5	-300.532	315.032
4	10.75	-202.52	213.27

After (X=4, Y=10.75)

$$w_0 = -49.006 + 0.6 * 213.27 * 4$$

$$w_1 = -6.496 + 0.6 * 213.27 * 1$$



If we evaluated the new function we get:

X	Y	Y _{out}	Error
2	6	1047.15	-1041.15
6	14.5	2898.518	-2884.02
4	10.75	1972.834	-1962.08

Notice that the network is oscillating wildly from one side to the other. This is due to the large learning rate of 0.6. It's not known if this network will eventually converge though it is unlikely, and even so it will take many epochs. This question demonstrates the importance of using small learning rates.